INF221 - Web Design & Development: REST

Isaac S. Mwakabira

Introductions

- Web APIs
- URI vs. URL
- API – Application Programming Interface, and examples
- REST(ful) APIs, and examples

APIs

- Communication - focus
- Abstraction
- Standardized
  - GraphQL - SOAP
  - REST - HTTP/HTTPs

APIs vs. SDKs

- Standard Dev. Kit (SDK)
- Toolbox or code that calls apis for you
- Languages: JAVA, PYTHON, etc
- Put Diagram Here

URI vs. URL

- URI – Universal **Resource** Identifier
  - Defined as any character string that identifies a resource on a particular server over the HTTP/s protocol
- URL – Unified **Resource** Locator
  - Specific type of URI
  - Locates an existing **RESOURCE** on the internet, the client (browser or tools like **POSTMAN**) makes a request to a server for the resource over HTTP/s protocol.
  - Defined as URIs that identify a resource by its location or by the means used to access it, rather than by a name or other attribute of the resource.
- Resource – uniquely identifiable object on the internet

URI vs. URL...

- General definition
  - scheme://host[:port#]/path/.../[;url-params][?query-string][#anchor]
  - Example:
    - http://127.0.0.1:80/api/users
    - https://127.0.0.1:80/api/users
    - https://www.cc.ac.mw/people/students
    - https://41.70.32.5/people/students

URI vs. URL...

- Parts of URL for HTTP/s protocol
  - Scheme – identifies the protocol used to access the resource on the Internet. Can be HTTP (without SSL) or HTTPS (with SSL)
  - Host – identifies where the resource is held, the host that holds the resource
  - Path - identifies the specific resource in the host that the web client wants to access.
  - Query string - provides a string of information that the resource can use. usually a string of name and value pairs ie. term=john
    - Name and value pairs are separated by an ampersand (&).
    - i.e. term=John&sort=DESC&limit=12
  - Anchor – identifies a fragment in a page resource i.e. especially if a resource is a web page
    - https://martinfowler.com/articles/richardsonMaturityModel.html#level1

API – Application Programming Interfaces

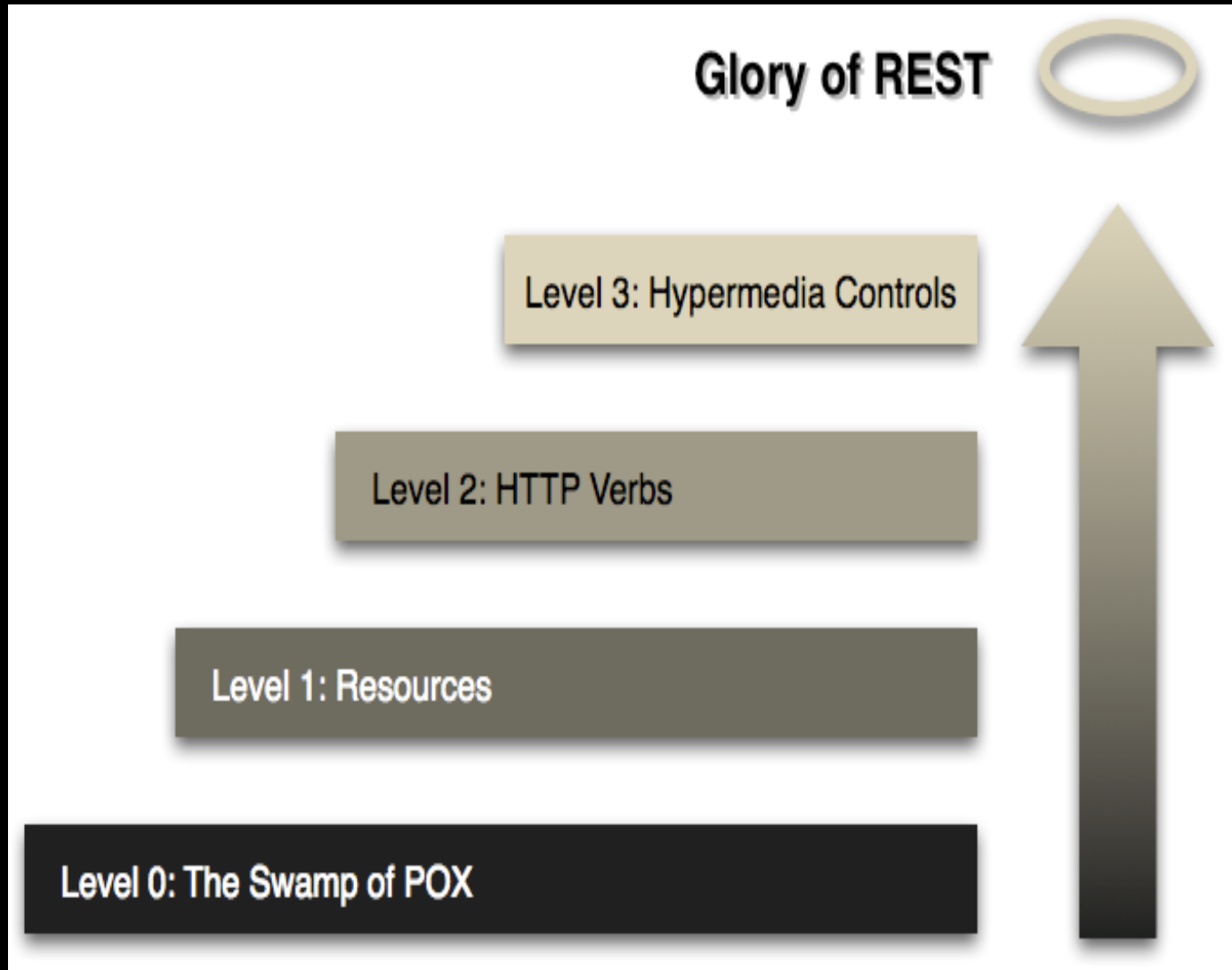
- **Contract** provided from one piece of software to another
  - Structured request and response
- Relies on stateless, client-server protocol, almost always HTTP
- Treats server objects as **resources** that can be created, updated or destroyed
- Can be designed by virtually using any web programming language and framework
  - Node (JS), NestJS (JS), Spring or Spring-boot (Java or Kotlin), .NET Framework (C#), Laravel (PHP)

RESTful APIs

- Representational State Transfer
  - Architecture style for designing networked applications
  - Describes design constraints that define the HTTP protocol
- Design Constraints. Discuss?
  - Addressability – unique & stable resource identifiers
  - Uniform Interface – resource interaction is through a uniform interaction provided by GET, POST, PATCH, DELETE, OPTIONS, HEAD
  - Stateless Interactions – client/server permanent sessions are not established to span multiple interactions
  - Self-Describing Messages – **responses** and **requests** contain both **data** and **meta-data**(description/information about data)
  - Hypermedia – relationship between **resources**

RESTful APIs

- Maturity Model. Discuss?
  - Is every API RESTful?
  - Steps towards REST



REST resources

- A resource is an object with a type, associated data, relation to other resources and a set of methods that operate on it.
- Similar to object instances in object-oriented programming.
- Each resource has its own address or URI—every interesting piece of information the server can provide is exposed as a resource.
- In deciding what resources are within your system, name them as nouns.
- RESTful URI should refer to a resource that is a thing instead of referring to an action.

Endpoint

Addresses/identifies a unique resource

The URI/URL where api/service can be accessed by a client application

Hypermedia

- HATEOAS
  - HyperText As The Engine Of Application State


```
{
    "description": "iPhone",
    "status": "IN_PROGRESS",
    "name": "Phone",
    "products": [],
    "_links": {
        "self": {
            "href": "http://127.0.0.1:8080/orders/6"
        },
        "all": {
            "href": "http://127.0.0.1:8080/orders"
        },
        "cancel": {
            "href": "http://127.0.0.1:8080/orders/6/cancel"
        },
        "complete": {
            "href": "http://127.0.0.1:8080/orders/6/complete"
        }
    }
},
{
    "description": "Windows 10 Pro",
    "status": "CANCELLED",
```

HTTP Verbs

The HTTP verbs comprise a major portion of our “uniform interface” constraint and provide us the action counterpart to the noun-based resource

HTTP Verbs (CRUD)

- GET: Retrieve data from specified resource
- POST: Submit data to be processed to a specified resource
- PUT: Update a specified resource
- DELETE: delete a specified resource

- HEAD: same as GET but does not return body
- OPTIONS: Returns the supported HTTP methods
- PATCH: Update partial resources

GET verb

The HTTP GET method is used to retrieve (or read) a representation of a resource.
Returns a representation in XML or JSON.

Examples:

1. GET http://www.example.com/customers/12345
2. GET http://www.example.com/customers/12345/orders
3. GET http://www.example.com/buckets/sample

According to the design of the HTTP specification, GET (along with HEAD) requests are used only to read data and not change it. Therefore, when used this way, they are considered safe.

PUT verb

Most-often utilized for update capabilities.

Examples:

1. PUT http://www.example.com/customers/12345
2. PUT http://www.example.com/customers/12345/orders/98765
3. PUT http://www.example.com/buckets/sample

PUT is not a safe operation, in that it modifies (or creates) state on the server,

POST verb

Most-often utilized for update capabilities.

Examples:

1. POST http://www.example.com/customers
2. POST http://www.example.com/customers/12345/orders

POST is neither safe or idempotent.

DELETE verb

It is used to delete a resource identified by a URI.

Examples:

1. DELETE http://www.example.com/customers/12345
2. DELETE http://www.example.com/customers/12345/orders
3. DELETE http://www.example.com/buckets/sample

DELETE is NOT safe.

Tools

Postman – stand alone application

JSON editor – broswer extension or as stand alone app

Browser

CURL

Summary

- URI vs. URL
- API – Application Programming Interface
- REST(ful) APIs, and examples

References

- https://dl.acm.org/doi/10.1145/514183.514185
- https://martinfowler.com/articles/richardsonMaturityModel.html
- Pautasso C. (2014) RESTful Web Services: Principles, Patterns, Emerging Technologies. In: Bouguettaya A., Sheng Q., Daniel F. (eds) Web Services Foundations. Springer, New York, NY. https://doi.org/10.1007/978-1-4614-7518-7_2